

作业 HW1 实验报告

姓名：汪嘉晨 学号：2353900 日期：2024 年 10 月 6 日

1. 涉及数据结构和相关背景

本次实验涉及的数据结构是线性表，线性表是由零个或多个数据元素组成的有限序列。若元素存在多个，则第一个元素无前驱，而最后一个元素无后继，其他元素都有且只有一个前驱和后继。线性表具有的特性：

- 1.有序性：元素之间具有明确的先后关系；
- 2.有限性：线性表中的元素个数是有限的；
- 3.均匀性：同一线性表中的元素必定具有相同的数据类型和长度（尽管不同线性表的数据元素可以不同）。

2. 实验内容

2.1 轮转数组

2.1.1 问题描述

给定一个整数顺序表 nums，将顺序表中的元素向右轮转 k 个位置，其中 k 是非负数。

2.1.2 基本要求

输入

第一行两个整数 n 和 k，分别表示 nums 的元素个数 n，和向右轮转 k 个位置；

第二行包括 n 个整数，为顺序表 nums 中的元素

输出`

轮转后的顺序表中的元素

2.1.3 数据结构设计

一开始使用了较为简单的直接设计，通过数组的方法来解决这个问题：

```
1. void rotateRight(int nums[], int n, int k) //旋转函数
2. {
3.     k = k % n;
4.     int rotated[100000];
5.     for (int i = 0; i < n; i++)
6.     {
7.         rotated[(i + k) % n] = nums[i];
8.     }
9.     for (int i = 0; i < n; i++)
10.    {
11.        nums[i] = rotated[i];
12.    }
13.}
14.int main() {
```

```

15.     int n, k;
16.     cin >> n >> k;
17.     int nums[100000];
18.     for (int i = 0; i < n; i++)
19.     {
20.         cin >> nums[i];
21.     }
22.     rotateRight(nums, n, k); //调用旋转函数
23.     for (int i = 0; i < n; i++)
24.     {
25.         cout << nums[i] << " ";
26.     }
27.     cout << endl;
28.     return 0;
29. }

```

后面意识到需要寻找更加简单（机智）的方法来解决问题，使得空间复杂度 $O(1)$ ，原地算法解决这个问题：

```

1. // 反转函数，反转数组中从 start 到 end 的元素
2. void reverse(int nums[], int start, int end) {
3.     while (start < end) {
4.         swap(nums[start], nums[end]);
5.         start++;
6.         end--;
7.     }
8. }
9. // 旋转函数，将数组 nums 向右旋转 k 个位置
10. void rotateRight(int nums[], int n, int k) {
11.     k = k % n; // 计算实际需要旋转的步数
12.     reverse(nums, 0, n - 1); // 反转整个数组
13.     reverse(nums, 0, k - 1); // 反转前 k 个元素
14.     reverse(nums, k, n - 1); // 反转剩余的元素
15. }
16. // 主函数，读取输入并调用旋转函数
17. int main() {
18.     int n, k;
19.     cin >> n >> k; // 读取数组长度和旋转步数
20.     int nums[100000];
21.     for (int i = 0; i < n; i++) {
22.         cin >> nums[i]; // 读取数组元素
23.     }
24.     rotateRight(nums, n, k); // 调用旋转函数
25.     for (int i = 0; i < n; i++) {
26.         cout << nums[i] << " "; // 输出旋转后的数组
27.     }

```

```
28.     return 0; }
```

2.1.4 功能说明（函数、类）

反转整个数组：`reverse(nums, 0, n - 1)`，将数组的所有元素反转。

反转前 k 个元素：`reverse(nums, 0, k - 1)`，将数组的前 k 个元素反转。

反转剩余的元素：`reverse(nums, k, n - 1)`，将数组的剩余元素（从 k 到末尾）反转。

时间复杂度： $O(n)$ ，每个元素被访问和交换三次。

空间复杂度： $O(1)$ ，只使用了常数级别的额外空间。

2.1.5 调试分析

问题一：数组越界问题。

解决方法：确保临时数组 `rotated` 的大小足够大，避免越界。

问题二：旋转步数大于数组长度。

解决方法：使用 $k = k \% n$ 计算实际需要旋转的步数。

2.1.6 总结和体会

学习了如何对数组进行旋转操作，并且理解了数组在内存中的存储方式。难点在于处理旋转步数大于数组长度的情况，以及确保数组不越界。通过优化算法，可以进一步提高效率，例如使用空间复杂度为 $O(1)$ 的原地算法。

难点在于如何去想 $O(1)$ 的问题，至于取余数应该很好去想到。

2.2 学生信息管理

2.2.1 问题描述

顺序表是指采用顺序存储结构的线性表，它利用内存中的一片连续存储区域存放表中的所有元素。可以根据需要对表中的所有数据进行访问，元素的插入和删除可以在表中的任何位置进行。顺序表的基本操作，包括顺序表的创建，第 i 个位置插入一个新的元素、删除第 i 个元素、查找某元素、顺序表的销毁。

2.2.2 基本要求

定义一个包含学生信息（学号，姓名）的顺序表，使其具有如下功能：(1) 根据指定学生个数，逐个输入学生信息；(2) 给定一个学生信息，插入到表中指定的位置；(3) 删除指定位置的学生记录；(4) 分别根据姓名和学号进行查找，返回此学生的信息；(5) 统计表中学生个数。

2.2.3 数据结构设计

使用了 C++ 中的 `struct` 和 `vector` 来存储和操作学生信息

```
1. struct Student {
2.     string number;
3.     string name;
4. };
5. // 自定义函数，将整数转换为字符串
6. string intToString(int n) {
7.     ostringstream ss;
```

```

8.     ss << n;
9.     return ss.str();
10. }
11. int main() {
12.     int n;
13.     cin >> n; // 读取学生数量
14.     vector<Student> students;
15.     students.reserve(10000); // 预留空间，避免频繁分配内存
16.     for (int i = 0; i < n; ++i) {
17.         Student s;
18.         cin >> s.number >> s.name; // 读取每个学生的学号和姓名
19.         students.push_back(s);
20.     }
21.     vector<string> outputs; // 存储输出结果

```

2.2.4 功能说明（函数、类）

主要在主函数中完成，涉及到插入删除查找销毁等：

```

1. while (getline(cin, line)) {
2.     istringstream iss(line);
3.     string command;
4.     iss >> command; // 读取命令
5.     if (command == "insert") {
6.         int position;
7.         Student new_student;
8.         iss >> position >> new_student.number >> new_student.n
           ame; // 读取插入位置和新学生信息
9.
10.        if (position < 1 || position > static_cast<int>(studen
           ts.size()) + 1) {
11.            outputs.push_back("-1"); // 插入位置不合法
12.        }
13.        else {
14.            students.insert(students.begin() + position - 1, n
           ew_student); // 插入新学生
15.            outputs.push_back("0"); // 插入成功
16.        }
17.    }
18.    else if (command == "remove") {
19.        int position;
20.        iss >> position; // 读取删除位置
21.        if (position < 1 || position > static_cast<int>(studen
           ts.size())) {
22.            outputs.push_back("-1"); // 删除位置不合法
23.        }

```

```

24.         else {
25.             students.erase(students.begin() + position - 1); /
                / 删除学生
26.             outputs.push_back("0"); // 删除成功
27.         }
28.     }
29.     else if (command == "check") {
30.         string type;
31.         iss >> type; // 读取检查类型 (name 或 no)
32.
33.         string to_find;
34.         iss >> to_find; // 读取要查找的姓名或学号
35.         bool found = false;
36.         for (size_t i = 0; i < students.size(); ++i) {
37.             if ((type == "name" && students[i].name == to_find
                ) ||
38.                 (type == "no" && students[i].number == to_find
                )) {
39.                 // 找到匹配的学生
40.                 string output = intToString(static_cast<int>(i
                ) + 1) + " " + students[i].number + " " + students[i].name;
41.                 outputs.push_back(output);
42.                 found = true;
43.                 break;
44.             }
45.         }
46.         if (!found) {
47.             outputs.push_back("-1"); // 未找到
48.         }
49.     }
50.     else if (command == "end") {
51.         outputs.push_back(intToString(static_cast<int>(student
                s.size()))); // 输出总数
52.         break;
53.     }
54. }
55. for (const auto& output : outputs) {
56.     cout << output << endl; // 统一输出结果
57. }

```

2.2.5 调试分析（遇到的问题和解决方法）

注：全部数值 ≤ 10000 ，元素位置从 1 开始。学生信息有重复数据（输入时未做检查），查找时只需返回找到的第一个。每个操作都在上一个操作的基础上完成。

题目中的温馨提示非常好，需要注意。

问题: 需要动态添加、删除和查询学生信息。

解决方法: 使用 `vector<Student>` 来存储学生信息。`vector` 是一个动态数组, 可以根据需要自动调整大小, 方便插入和删除操作。(见前面)

2.2.6 总结和体会

动态数组管理: 使用 `vector` 动态管理学生信息, 需要理解 `vector` 的插入、删除和遍历操作。这道题折腾的挺麻烦, `vector` 在高程课上没有学到, 也是用到了 copilot 的帮助。

2.3 一元多项式的相加和相乘

2.3.1 问题描述

一元多项式是有序线性表的典型应用, 用一个长度为 m 且每个元素有两个数据项 (系数项和指数项) 的线性表 $((p_1, e_1), (p_2, e_2), \dots, (p_m, e_m))$ 可以唯一地表示一个多项式。本题实现多项式的相加和相乘运算。本题输入保证是按照指数项递增有序的。

2.3.2 基本要求

输入:

第 1 行一个整数 m , 表示第一个一元多项式的长度

第 2 行有 $2m$ 个整数, $p_1\ e_1\ p_2\ e_2\ \dots$, 中间以空格分割, 表示第 1 个多项式系数和指数

第 3 行一个整数 n , 表示第二个一元多项式的项数

第 4 行有 $2n$ 个整数, $p_1\ e_1\ p_2\ e_2\ \dots$, 中间以空格分割, 表示第 2 个多项式系数和指数

第 5 行一个整数, 若为 0, 执行加法运算并输出结果, 若为 1, 执行乘法运算并输出结果; 若为 2, 输出一行加法结果和一行乘法的结果。

输出:

运算后的多项式链表, 要求按指数从小到大排列

当运算结果为 0 0 时, 不输出。

2.3.3 数据结构设计

使用了结构体和类 (有点长):

```
1. struct Node {
2.     int coef; // 系数
3.     int exp;  // 指数
4.     Node* next;
5.     Node(int c, int e) : coef(c), exp(e), next(nullptr) {}
6. };
7.
8. class Polynomial {
9. public:
10.     Polynomial() : head(nullptr) {}
11.
12.     void insert(int coef, int exp) {
13.         Node* newNode = new Node(coef, exp);
14.         if (!head || head->exp > exp) {
```

```

15.         newNode->next = head;
16.         head = newNode;
17.     }
18.     else {
19.         Node* current = head;
20.         while (current->next && current->next->exp < exp)
21.         {
22.             current = current->next;
23.         }
24.         newNode->next = current->next;
25.         current->next = newNode;
26.     }
27.
28. Polynomial add(Polynomial& other) {
29.     Polynomial result;
30.     Node* p1 = head;
31.     Node* p2 = other.head;
32.     while (p1 && p2) {
33.         if (p1->exp == p2->exp) {
34.             int sumCoef = p1->coef + p2->coef;
35.             if (sumCoef != 0) {
36.                 result.insert(sumCoef, p1->exp);
37.             }
38.             p1 = p1->next;
39.             p2 = p2->next;
40.         }
41.         else if (p1->exp < p2->exp) {
42.             result.insert(p1->coef, p1->exp);
43.             p1 = p1->next;
44.         }
45.         else {
46.             result.insert(p2->coef, p2->exp);
47.             p2 = p2->next;
48.         }
49.     }
50.     while (p1) {
51.         result.insert(p1->coef, p1->exp);
52.         p1 = p1->next;
53.     }
54.     while (p2) {
55.         result.insert(p2->coef, p2->exp);
56.         p2 = p2->next;
57.     }

```

```

58.         return result;
59.     }
60.     Polynomial multiply(Polynomial& other) {
61.         Polynomial result;
62.         for (Node* p1 = head; p1; p1 = p1->next) {
63.             Polynomial temp;
64.             for (Node* p2 = other.head; p2; p2 = p2->next) {
65.                 temp.insert(p1->coef * p2->coef, p1->exp + p2-
>exp);
66.             }
67.             result = result.add(temp);
68.         }
69.         return result;
70.     }
71.
72.     void print() {
73.         Node* current = head;
74.         bool first = true;
75.         while (current) {
76.             if (!first) {
77.                 cout << " ";
78.             }
79.             cout << current->coef << " " << current->exp;
80.             first = false;
81.             current = current->next;
82.         }
83.         cout << endl;
84.     }
85.
86. private:
87.     Node* head;
88. };

```

2.3.4 功能说明（函数、类）

可以进行多项式的加法和乘法，并实现最后的输出：见上方黄底

2.3.5 调试分析（遇到的问题和解决方法）

一开始只得到了 63 分（最后成了 100 分）并显示超时，后面通过 ai 分析得到了答案：

出现“Time Limit Exceeded”错误通常是因为代码在处理大数据集时效率不高。对于多项式相乘的实现，当前的代码复杂度较高，可能会导致在处理较大输入时超时。我们可以优化多项式相乘的实现，以提高效率。

优化后的 C++ 代码，使用了哈希表来存储中间结果，从而减少重复计算：

```

1. Polynomial multiply(Polynomial& other) {
2.     unordered_map<int, int> resultMap;

```

```

3.     for (Node* p1 = head; p1; p1 = p1->next) {
4.         for (Node* p2 = other.head; p2; p2 = p2->next) {
5.             int newExp = p1->exp + p2->exp;
6.             int newCoef = p1->coef * p2->coef;
7.             resultMap[newExp] += newCoef;
8.         }
9.     }
10.
11.    Polynomial result;
12.    for (const auto& term : resultMap) {
13.        if (term.second != 0) {
14.            result.insert(term.second, term.first);
15.        }
16.    }
17.    return result;
18. }

```

2.3.6 总结和体会

哈希表 (`unordered_map`) 的精华部分在于它用于存储多项式相乘的中间结果，从而提高了计算效率。具体来说，哈希表用于存储每个指数对应的系数之和，避免了重复计算。平常应该多多注意效率与复杂性。

2.4 求级数

2.4.1 问题描述

求级数

输入：

若干行，在每一行中给出整数 N 和 A 的值，($1 \leq N \leq 150$, $0 \leq A \leq 15$)

对于 20% 的数据，有 $1 \leq N \leq 12$, $0 \leq A \leq 5$

对于 40% 的数据，有 $1 \leq N \leq 18$, $0 \leq A \leq 9$

对于 100% 的数据，有 $1 \leq N \leq 150$, $0 \leq A \leq 15$

从右上方下载 `p95.py` 并运行以生成随机测试数据

输出：

对于每一行，在一行中输出级数 $A + 2A^2 + 3A^3 + \dots + NA^N = \sum_{i=1}^N iA^i$ 的整数值

2.4.2 基本要求

高精度

2.4.3 数据结构设计

1. 大整数加法： `add` 函数将两个大整数字符串相加。
2. 大整数乘法： `multiply` 函数将一个大整数字符串乘以一个小整数。

3. **计算次方**: `power` 函数计算 `a` 的 `i` 次方, 返回一个大整数字符串。

4. **主函数**: 读取 `n` 和 `a` 的值, 计算级数并输出结果。

一开始直接用 `long long`, 提交时发现超过了 `long long` 承受的范围, 于是考虑通过字符串、`string` 来解决问题

2.4.4 功能说明 (函数、类)

```
1. // 定义一个函数来将两个大整数相加
2. string add(const string& num1, const string& num2) {
3.     string result;
4.     int carry = 0, sum = 0;
5.     int len1 = num1.size(), len2 = num2.size();
6.     int i = len1 - 1, j = len2 - 1;
7.     while (i >= 0 || j >= 0 || carry) {
8.         sum = carry;
9.         if (i >= 0) sum += num1[i--] - '0';
10.        if (j >= 0) sum += num2[j--] - '0';
11.        carry = sum / 10;
12.        result.push_back(sum % 10 + '0');
13.    }
14.    reverse(result.begin(), result.end());
15.    return result;
16.}
17. // 定义一个函数来将一个大整数乘以一个小整数
18. string multiply(const string& num, int x) {
19.     string result;
20.     int carry = 0, product = 0;
21.     int len = num.size();
22.     for (int i = len - 1; i >= 0; i--) {
23.         product = (num[i] - '0') * x + carry;
24.         carry = product / 10;
25.         result.push_back(product % 10 + '0');
26.     }
27.     while (carry) {
28.         result.push_back(carry % 10 + '0');
29.         carry /= 10;
30.     }
31.     reverse(result.begin(), result.end());
32.     return result;
33.}
34. // 定义一个函数来计算 a 的 i 次方
35. string power(int a, int i) {
36.     string result = "1";
37.     for (int j = 0; j < i; j++) {
38.         result = multiply(result, a);
```

```
39.     }
40.     return result;
41. }
```

2.4.5 调试分析（遇到的问题 and 解决方法）

问题：long long 都不够用，于是考虑上个学期的高程课（大数相加）

```
/*#include<iostream>
using namespace std;

int main()
{
    long long num = 0;
    int n, a;
    cin >> n >> a;
    long long ai = 1; // 用于存储a的i次方
    for (int i = 1; i <= n; i++)
    {
        ai *= a; // 计算a的i次方
        num += i * ai;
    }
    cout << num << endl;
    return 0;
}
*/
```

2.1.6 总结和体会

通过使用字符串来处理大整数运算，可以：

- 1.避免溢出：使用字符串来表示大整数，可以避免传统整数类型在处理大数时的溢出问题；
- 2.通用性强：字符串处理大整数的方法不依赖于特定的编译器或平台，具有很强的通用性；
- 3.灵活性：字符串可以表示任意长度的整数，适用于各种需要处理大数的场景。

2.5 扑克牌游戏

2.5.1 问题描述

扑克牌有 4 种花色：黑桃（Spade）、红心（Heart）、梅花（Club）、方块（Diamond）。每种花色有 13 张牌，编号从小到大为：A,2,3,4,5,6,7,8,9,10,J,Q,K。

对于一个扑克牌堆，定义以下 4 种操作命令：

- 1) 添加（Append）：添加一张扑克牌到牌堆的底部。如命令“Append Club Q”表示添加一张梅花 Q 到牌堆的底部。
- 2) 抽取（Extract）：从牌堆中抽取某种花色的所有牌，按照编号从小到大进行排序，并放到牌堆的顶部。如命令“Extract Heart”表示抽取所有红心牌，排序之后放到牌堆的顶部。

3)反转 (Revert): 使整个牌堆逆序。

4)弹出 (Pop): 如果牌堆非空, 则除去牌堆顶部的第一张牌, 并打印该牌的花色和数字; 如果牌堆为空, 则打印 NULL。

初始时牌堆为空。输入 n 个操作命令 ($1 \leq n \leq 200$), 执行对应指令。所有指令执行完毕后打印牌堆中所有牌花色和数字 (从牌堆顶到牌堆底), 如果牌堆为空, 则打印 NULL

注意: 每种花色和编号的牌数量不限。

对于 20%的数据, $n \leq 20$, 有 Append、Pop 指令

对于 40%的数据, $n \leq 50$, 有 Append、Pop、Revert 指令

对于 100%的数据, $n \leq 200$, 有 Append、Pop、Revert、Extract 指令

从右上方下载 p98.py 并运行以生成随机测试数据

2.5.2 基本要求

输入描述

第一行输入一个整数 n , 表示命令的数量。

接下来的 n 行, 每一行输入一个命令。

输出描述

输出若干行, 每次收到 Pop 指令后输出一行 (花色和数字或 NULL), 最后将牌堆中的牌从牌堆顶到牌堆底逐一输出 (花色和数字), 若牌堆为空则输出 NULL

2.5.3 数据结构设计

```
1. // 定义扑克牌结构体
2. struct Card {
3.     int suit; // 0=黑桃, 1=红心, 2=梅花, 3=方块
4.     int rank; // 1=A, 2-10, 11=J, 12=Q, 13=K
5.     Card* prev; // 前一张牌
6.     Card* next; // 后一张牌
7.     // 构造函数
8.     Card(int s, int r) : suit(s), rank(r), prev(NULL), next(NULL) {}
9. };
```

2.5.4 功能说明 (函数、类)

```
1. // 将花色字符串转换为整数
2. int suitToInt(const string& suitStr) {
3.     if (suitStr == "Spade") return 0;
4.     if (suitStr == "Heart") return 1;
5.     if (suitStr == "Club") return 2;
6.     if (suitStr == "Diamond") return 3;
7.     return -1; // 无效花色
8. }
9.
10. // 将牌面字符串转换为整数
11. int rankToInt(const string& rankStr) {
12.     if (rankStr == "A") return 1;
```

```
13.     if (rankStr == "J") return 11;
14.     if (rankStr == "Q") return 12;
15.     if (rankStr == "K") return 13;
16.     else return atoi(rankStr.c_str());
17. }
18.
19. // 将整数转换为花色字符串
20. string intToSuit(int suitInt) {
21.     if (suitInt == 0) return "Spade";
22.     if (suitInt == 1) return "Heart";
23.     if (suitInt == 2) return "Club";
24.     if (suitInt == 3) return "Diamond";
25.     return "";
26. }
27.
28. // 将整数转换为牌面字符串
29. string intToRank(int rankInt) {
30.     if (rankInt == 1) return "A";
31.     if (rankInt == 11) return "J";
32.     if (rankInt == 12) return "Q";
33.     if (rankInt == 13) return "K";
34.     else {
35.         char buf[4];
36.         sprintf(buf, "%d", rankInt);
37.         return string(buf);
38.     }
39. }
40.
41. // 修剪字符串，去除首尾空白字符，包括空格、制表符、回车符和换行符
42. string trim(const string& s) {
43.     size_t first = s.find_first_not_of(" \t\r\n");
44.     if (first == string::npos)
45.         return "";
46.     size_t last = s.find_last_not_of(" \t\r\n");
47.     return s.substr(first, (last - first + 1));
48. }
49.
50. // 将一张牌添加到牌堆底部
51. void append(Card*& head, Card*& tail, Card* newCard) {
52.     if (tail == NULL) {
53.         head = tail = newCard;
54.     }
55.     else {
56.         tail->next = newCard;
```

```

57.         newCard->prev = tail;
58.         tail = newCard;
59.     }
60. }
61.
62. // 从牌堆顶部弹出一张牌
63. Card* pop(Card*& head, Card*& tail) {
64.     if (head == NULL) {
65.         return NULL;
66.     }
67.     else {
68.         Card* removedCard = head;
69.         head = head->next;
70.         if (head != NULL) {
71.             head->prev = NULL;
72.         }
73.         else {
74.             tail = NULL;
75.         }
76.         removedCard->next = NULL;
77.         removedCard->prev = NULL;
78.         return removedCard;
79.     }
80. }
81.
82. // 逆转整个牌堆
83. void revert(Card*& head, Card*& tail) {
84.     Card* current = head;
85.     Card* temp = NULL;
86.     while (current != NULL) {
87.         temp = current->prev;
88.         current->prev = current->next;
89.         current->next = temp;
90.         current = current->prev; // 因为 prev 和 next 已经
            交换
91.     }
92.     // 交换头尾指针
93.     temp = head;
94.     head = tail;
95.     tail = temp;
96. }
97.
98. // 比较两张牌的大小，用于排序
99. bool compareCards(const Card* a, const Card* b) {

```

```

100.     return a->rank < b->rank;
101. }
102.
103. // 抽取某一花色的所有牌
104. void extract(Card*& head, Card*& tail, int suit) {
105.     vector<Card*> extractedCards;
106.     Card* current = head;
107.     while (current != NULL) {
108.         if (current->suit == suit) {
109.             Card* toRemove = current;
110.             current = current->next;
111.             // 从牌堆中移除该牌
112.             if (toRemove->prev != NULL) {
113.                 toRemove->prev->next = toRemove->next;
114.             }
115.             else {
116.                 head = toRemove->next;
117.             }
118.             if (toRemove->next != NULL) {
119.                 toRemove->next->prev = toRemove->prev;
120.             }
121.             else {
122.                 tail = toRemove->prev;
123.             }
124.             toRemove->prev = toRemove->next = NULL;
125.             extractedCards.push_back(toRemove);
126.         }
127.         else {
128.             current = current->next;
129.         }
130.     }
131.     // 按照牌面从小到大排序
132.     sort(extractedCards.begin(), extractedCards.end(), compareCards);
133.     // 将抽取的牌放到牌堆顶部
134.     for (int i = extractedCards.size() - 1; i >= 0; --i) {
135.         Card* card = extractedCards[i];
136.         if (head == NULL) {
137.             head = tail = card;
138.         }
139.         else {
140.             card->next = head;
141.             head->prev = card;
142.             head = card;

```

```
143.             card->prev = NULL;
144.         }
145.     }
146. }
```

2.1.5 调试分析（遇到的问题解决方法）

这道题和前面的题真的很像，综合了一下结束。

2.1.6 总结和体会

融会贯通甚好！当然还需要不断学习，感谢人工智能的纠错与更改！

3. 实验总结

线性表作为最基本的数据结构之一，通过对其定义、特性（如顺序性、有限性、元素类型相同等）的学习，我对数据结构有了更加深入的理解。认识到数据结构不仅仅是数据的简单存储，更是数据之间关系的体现，这种关系决定了数据的组织和处理方式。线性表作为数据结构的基础，其背后隐藏着许多更深层次的知识和技术。在学习过程中，我不断发现新的问题和挑战，这激发了我持续学习和探索的热情。